



# ALMOST SHORTEST PATH



# PROBLEMA E CONTEXTO

JUNE 2026

Encontrar o caminho mais curto entre um ponto de partida e um ponto de destino, dados um conjunto de pontos e os comprimentos das rotas que os conectam, é um problema já bastante conhecido e até faz parte do nosso dia a dia, pois programas de cálculo de caminho mais curto estão amplamente disponíveis hoje em dia.

A maioria das pessoas gosta muito desses aplicativos, pois facilitam suas vidas. Bem, talvez não tanto assim.

Agora que quase todos têm acesso a dispositivos de navegação GPS capazes de calcular os caminhos mais curtos, a maioria das rotas que formam o caminho mais curto está ficando mais lenta devido ao tráfego intenso. Como a maioria das pessoas tenta seguir o mesmo caminho, não vale mais a pena seguir essas indicações.

Com isso em mente, seu chefe pede que você desenvolva um novo aplicativo ao qual somente ele terá acesso, economizando tempo sempre que tiver uma reunião ou algum evento urgente. Ele pede que o programa não encontre o caminho mais curto, mas sim o caminho quase mais curto. Ele define o caminho quase mais curto como o caminho mais curto que vai de um ponto de partida a um ponto de destino, de forma que nenhuma rota entre dois pontos consecutivos pertença a qualquer caminho mais curto do ponto de partida ao destino.

# Algoritmo de Dijkstra

O ALGORITMO DE DIJKSTRA É UM MÉTODO PARA ENCONTRAR O CAMINHO MAIS CURTO ENTRE UM PONTO DE ORIGEM E TODOS OS OUTROS VÉRTICES DE UM GRAFO COM PESOS NÃO NEGATIVOS NAS ARESTAS.

### Passo 1: Inicialização

Tabela de distâncias:

| Vértice | Distância |
|---------|-----------|
| A       | 0         |
| B       | $\infty$  |
| C       | $\infty$  |
| D       | $\infty$  |

### Passo 4: Processar C

Vizinho:

$D = 2 + 8 = 10$

Atualizamos:

| Vértice | Distância |
|---------|-----------|
| A       | 0         |
| B       | 4         |
| C       | 2         |
| D       | 10        |

### Passo 2: Processar A

Vizinhos de A:

- B:  $0 + 4 = 4$
- C:  $0 + 2 = 2$

Atualizamos:

| Vértice | Distância |
|---------|-----------|
| A       | 0         |
| B       | 4         |
| C       | 2         |
| D       | $\infty$  |

### Passo 3: Escolher menor distância

Entre os não visitados:

B = 4  
C = 2  
D =  $\infty$

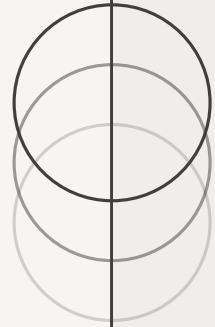
Escolhemos C.

### Passo 5: Escolher menor distância

Entre os não visitados:

B = 4  
D = 10

Escolhemos B.



## Passo 6: Processar B

JUNE 2026

Para C:

$$4 + 1 = 5$$

Mas C já tem distância 2, então não atualiza.

Para D:

$$4 + 5 = 9$$

Como  $9 < 10$ :

Atualizamos D.

| Vértice | Distância |
|---------|-----------|
| A       | 0         |
| B       | 4         |
| C       | 2         |
| D       | 9         |

$$\text{DIST}(V) > \text{DIST}(U) + W(U,V)$$

# ALMOST SHORTEST PATH EM PARALELO COM DIJKSTRA

JUNE 2026



o algoritmo de dijkstra calcula o menor caminho possível entre um vértice e outro, e desconsidera todas as arestas presentes no menor caminho. O algoritmo é executado novamente, para assim pegar o segundo caminho mais curto entre um vértice e outro. No caso do problema, serve para evitar congestionamento no trânsito, já que todo mundo opta por ir pelo menor caminho possível.



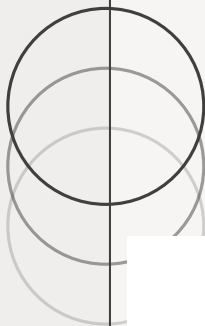


# InPut

A ENTRADA CONTÉM VÁRIOS CASOS DE TESTE. A PRIMEIRA LINHA DE UM CASO DE TESTE CONTÉM DOIS INTEIROS  $N$  ( $2 \leq N \leq 500$ ) E  $M$  ( $1 \leq M \leq 104$ ), SEPARADOS POR UM ÚNICO ESPAÇO, INDICANDO RESPECTIVAMENTE O NÚMERO DE PONTOS NO MAPA E O NÚMERO DE ROTAS UNIDIRECIONAIS EXISTENTES QUE CONECTAM DOIS PONTOS DIRETAMENTE. CADA PONTO É IDENTIFICADO POR UM INTEIRO ENTRE 0 E  $N - 1$ . A SEGUNDA LINHA CONTÉM DOIS INTEIROS  $S$  E  $D$ , SEPARADOS POR UM ÚNICO ESPAÇO, INDICANDO RESPECTIVAMENTE OS PONTOS DE PARTIDA E DESTINO ( $S \neq D$ ;  $0 \leq S, D < N$ ). CADA UMA DAS SEGUINTE  $M$  LINHAS CONTÉM TRÊS INTEIROS  $U, V, P$  ( $U \neq V$ ;  $0 \leq U, V < N$ ;  $1 \leq P \leq 103$

# OutPut

PARA CADA CASO DE TESTE NA ENTRADA, SEU PROGRAMA DEVE IMPRIMIR UMA ÚNICA LINHA, CONTENDO '-1' SE NÃO FOR POSSÍVEL ATENDER AOS REQUISITOS, OU UM INTEIRO REPRESENTANDO O COMPRIMENTO DO CAMINHO QUASE MAIS CURTO ENCONTRADO.



# Exemplo

JUNE 2026

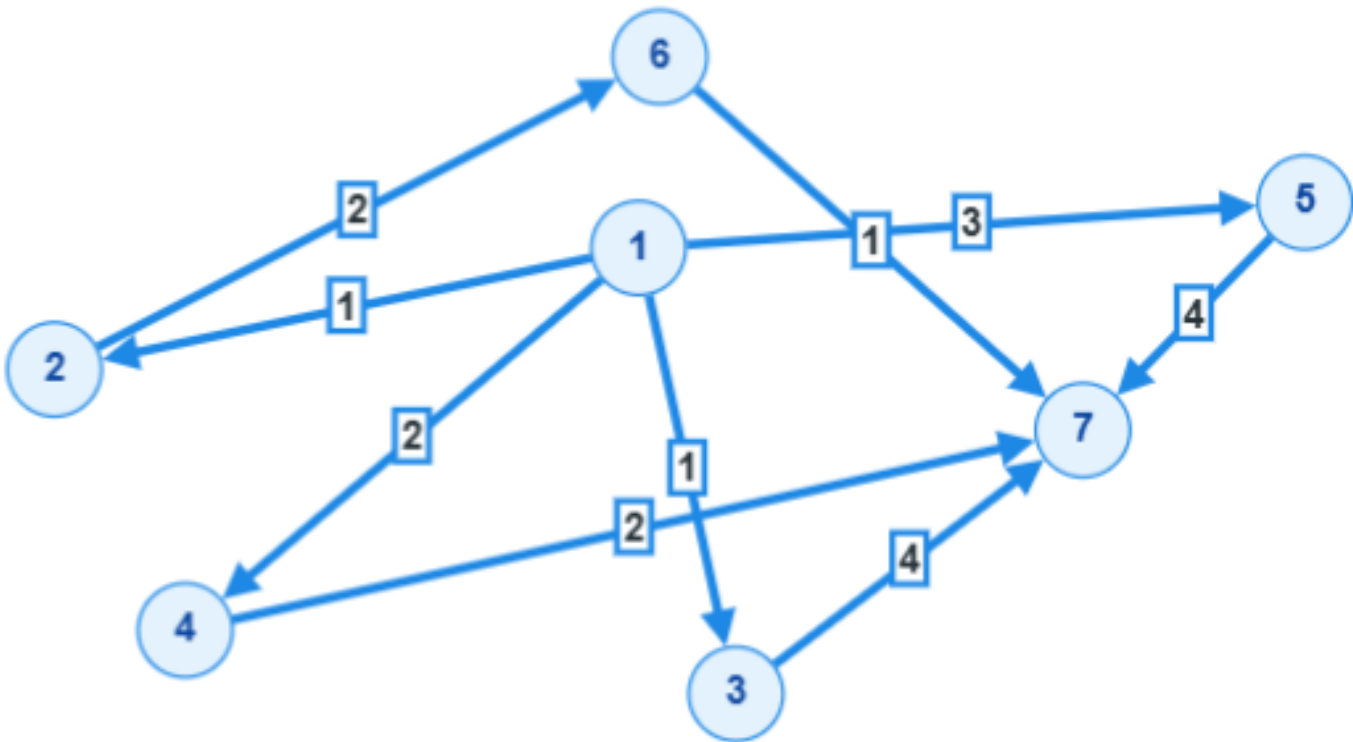
ENTRADA

```
7 9
0 6
0 1 1
0 2 1
0 3 2
0 4 3
1 5 2
2 6 4
3 6 2
4 6 4
5 6 1
4 6
0 2
0 1 1
1 2 1
1 3 1
3 2 1
2 0 3
3 0 2
6 8
0 1
0 1 1
0 2 2
0 3 3
2 5 3
3 4 2
4 1 1
5 1 1
3 0 1
0 0
```

SAÍDA

**Sample Output**

```
5
-1
6
```



# Next Month

## Caso 1

Menores caminhos de 0 até 6:

$0 \rightarrow 1 \rightarrow 5 \rightarrow 6 = 4$   
 $0 \rightarrow 3 \rightarrow 6 = 4$

Removendo as arestas pertencentes a qualquer menor caminho:

$0 \rightarrow 1$   
 $1 \rightarrow 5$   
 $5 \rightarrow 6$   
 $0 \rightarrow 3$   
 $3 \rightarrow 6$

Sobra:

$0 \rightarrow 2 \rightarrow 6 = 5$

Resultado:

5



## Caso 2

Menor caminho:

$0 \rightarrow 1 \rightarrow 2 = 2$

Removendo:

$0 \rightarrow 1$   
 $1 \rightarrow 2$

O vértice 0 fica sem caminho para 2.

Resultado:

-1

## Caso 3

Menor caminho:

$0 \rightarrow 1 = 1$

Removendo:

$0 \rightarrow 1$

Restam:

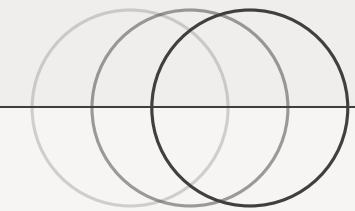
$0 \rightarrow 2 \rightarrow 5 \rightarrow 1 = 6$   
 $0 \rightarrow 3 \rightarrow 4 \rightarrow 1 = 6$

Resultado:

6



# Complexidade e Casos Especiais

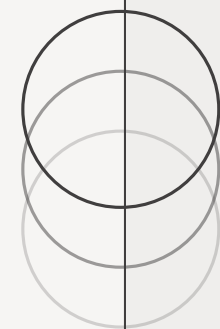


## Casos especiais

- Não existe caminho inicial  $\rightarrow -1$
- Múltiplos caminhos mínimos  $\rightarrow$  remover todos
- Não existe caminho após a remoção  $\rightarrow -1$

## Complexidade

- Tempo:  $O((V + E) \log V)$
- Memória:  $O(V + E)$



# THANKYOU

Gustavo Leon - 2410430